

МОСКОВСКИЙ ФИЗИКО–ТЕХНИЧЕСКИЙ ИНСТИТУТ

АРХИТЕКТУРА ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ
ЗАДАНИЕ №2



ФИО: Лазарь Владислав Игоревич

Группа: Б01-207а

Почта: lazar.v@phystech.edu

Вариант №1: alu_register

Долгопрудный, 2026 г.

1. Функциональное описание.

Разрабатываемый модуль `alu_register` представляет собой арифметико-логическое устройство с интегрированным выходным регистром (D -триггером). Модуль предназначен для выполнения набора из восьми арифметических и побитовых операций над входными операндами и сохранения результата в синхронном буфере:

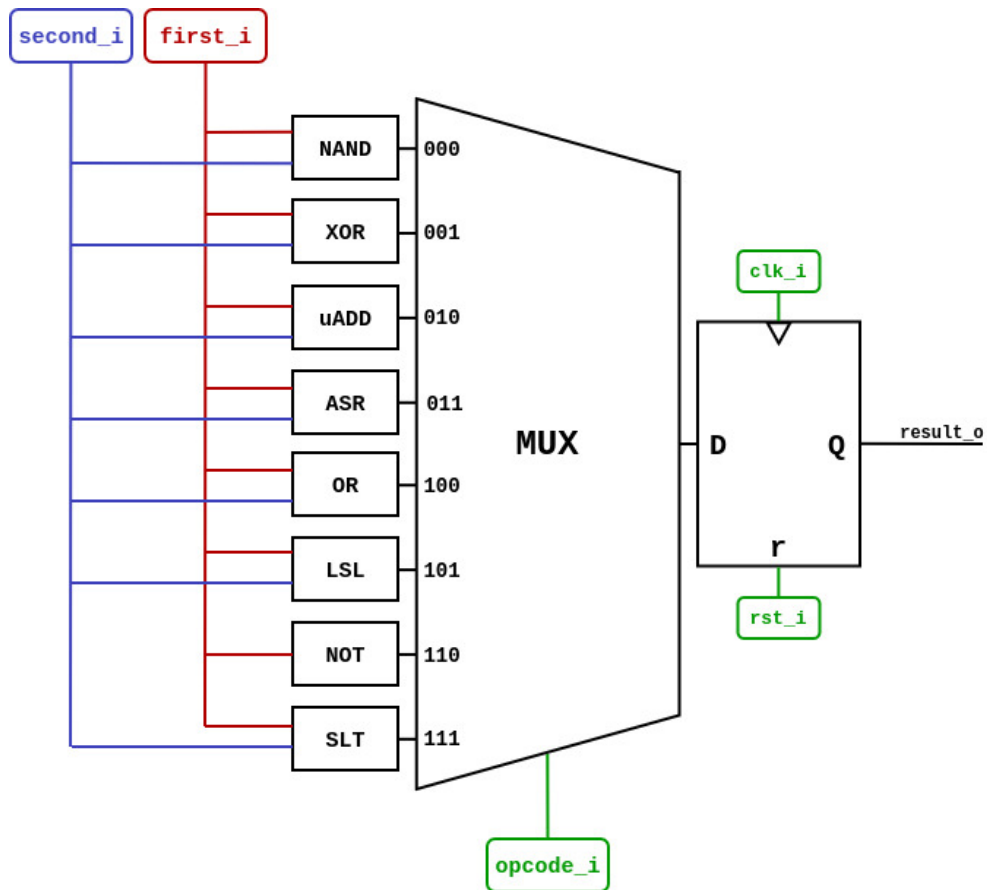
<code>opcode_i</code>	Операция над <code>first_i</code> и <code>second_i</code>	Описание
<code>3'b000</code>	$\text{result} = \neg(\text{first_i} \wedge \text{second_i})$	Побитовое НЕ-И (NAND)
<code>3'b001</code>	$\text{result} = \text{first_i} \oplus \text{second_i}$	Побитовое исключающее ИЛИ (XOR)
<code>3'b010</code>	$\text{result} = \text{first_i} + \text{second_i}$	Сложение (без знака)
<code>3'b011</code>	$\text{result} = \text{first_i} \gg_{\text{arith}} \text{second_i}$	Арифметический (знаковый) сдвиг <code>first_i</code> вправо на <code>second_i</code>
<code>3'b100</code>	$\text{result} = \text{first_i} \vee \text{second_i}$	Побитовое ИЛИ (OR)
<code>3'b101</code>	$\text{result} = \text{first_i} \ll \text{second_i}$	Логический сдвиг <code>first_i</code> влево на <code>second_i</code>
<code>3'b110</code>	$\text{result} = \neg \text{first_i}$	Побитовое НЕ для <code>first_i</code>
<code>3'b111</code>	$\text{result} = (\text{first_i} < \text{second_i})$	Сравнение операндов на «меньше»

Таблица 1: Математическое описание операций.

У модуля есть некоторые функциональные особенности:

- Ширина обрабатываемых данных задается параметром `WIDTH`, что позволит нам использовать модуль в системах с различной разрядностью.
- Выходная шина `result_o` всегда отображает текущее состояние внутреннего D -триггера.
- Результат операции, вычисленный на основе входных данных `first_i`, `second_i` и `opcode_i` в текущем такте, фиксируется в регистре и подается на выход `result_o` только в начале следующего тактового периода.
- Все операции записи в выходной регистр выполняются строго по положительному фронту тактового сигнала `clk_i`.

2. Структурная схема.



Здесь используются следующие сокращения:

- uADD — Unsigned Addition;
- ASR — Arithmetic Shift Right;
- LSL — Logical Shift Left;
- SLT — Set Less Than.

3. Параметры.

В модуле `alu_register` используется один параметр конфигурации — `WIDTH`, который имеет тип `integer` и отвечает за определение разрядности (ширины) обрабатываемых данных `first_i`, `second_i` и `result_o`.

В качестве значения `WIDTH` может выступать любое целое положительное число. Наиболее часто используются значения, кратные степени двойки (например, *1, 8, 16, 32 ...*). По умолчанию в модуле установлено значение *8*.

4. Порты.

Название	Ширина (бит)	Направление	Описание
clk_i	1	Вход	Тактовый сигнал. Запись в регистр происходит по положительному фронту.
rst_i	1	Вход	Синхронный сброс активного высокого уровня. Обнуляет result_o .
first_i	WIDTH	Вход	Шина первого операнда для арифметико-логических операций.
second_i	WIDTH	Вход	Шина второго операнда для арифметико-логических операций.
opcode_i	3	Вход	Код выполняемой операции.
result_o	WIDTH	Выход	Шина результата. Значение обновляется с задержкой в один такт.

5. Тактирование и сброс.

Тактирование осуществляется внешним сигналом **clk_i**. Запись данных из АЛУ во внутренний регистр происходит строго по положительному фронту тактового импульса.

В модуле реализован *синхронный сброс* по высокому уровню сигнала **rst_i**. Для приведения модуля в исходное состояние необходимо подать на вход **rst_i** логическую единицу. Так как сброс является синхронным, обнуление выходного регистра произойдет не мгновенно, а в момент прихода следующего положительного фронта сигнала **clk_i**. После этого значение на выходе **result_o** установится в 0 и будет удерживаться до тех пор, пока сигнал **rst_i** не будет снят (т. е. установлен в 0) и не придет следующий тактовый импульс для записи результата новой операции.

6. Описание сценариев тестирования.

Тестирование будет включать в себя следующие этапы:

- Генерация тактового сигнала **clk_i** с периодом 10 нс.
- Установка сигнала **rst_i** в 1. Проверка того, что по приходу положительного фронта **clk_i** значение на выходе **result_o** становится равным 0.
- Подача операндов и кода операции (например, сложения 3'b010) и контроль того, что результат вычисления появляется на шине **result_o** на следующем такте после подачи данных.
- Последовательная подача всех восьми кодов операций из таблицы функций.
- Тестирование операций с нулевыми операндами и значениями, состоящими из всех единиц (WIDTH{1'b1}).
- Проверка того, что активация **rst_i** в любой момент выполнения операций приводит к обнулению выходного регистра на следующем такте.

7. Имплементация модуля на Verilog и тестирование.

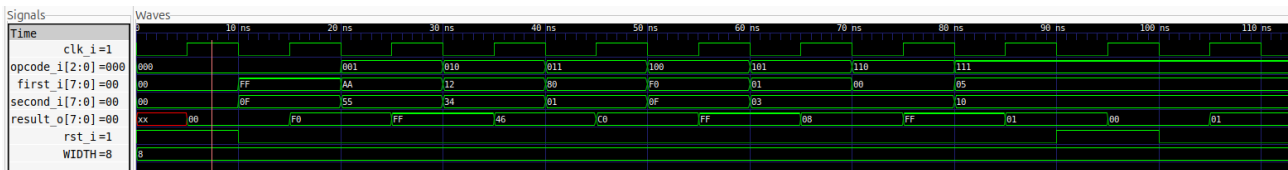
Имплементацию модуля `alu_register` можно найти в файле `./alu_register.v`.

Тестирование модуля `alu_register` можно найти в файле `./alu_register_tb.v`.

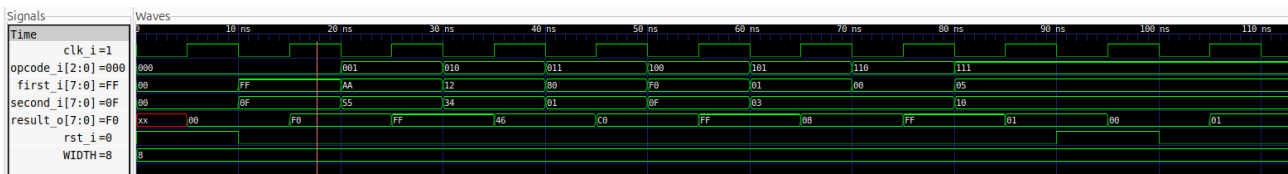
8. Временные диаграммы, полученные с помощью разработанного testbench.

Для просмотра временных диаграмм использовалась программа GTKWave, а для компиляции — Icarus Verilog.

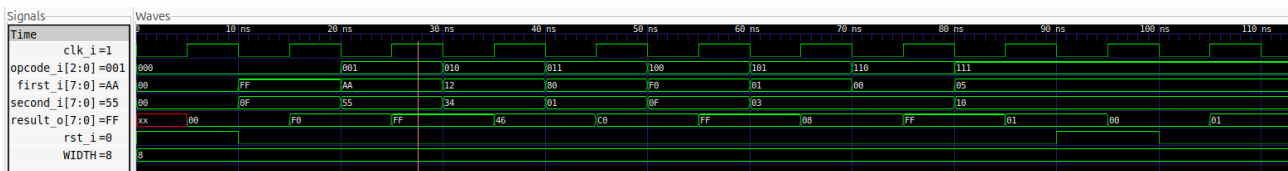
Со всеми временными диаграммами в нормальном качестве и размере можно ознакомиться в `./testing_alu_register`. Вот некоторые из них:



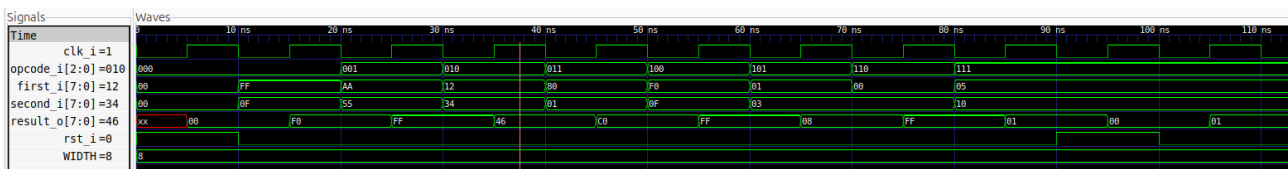
Тест №1 — инициализация и синхронный сброс.



Тест №2 — NAND: $(0xFF \wedge 0x0F) = 0xF0$.



Тест №3 — XOR: $0xAA \oplus 0x55 = 0xFF$.



Тест №4 — uADD: $0x12 + 0x34 = 0x46$.